WPT Day 7 Morning

Tue 27 Oct 26 · The `var name` / `window.name` hoisting trap, three kinds of Node modules, npm as a CLI package manager, `package.json` / `node_modules` / lockfile, local vs global installs, and CommonJS vs ECMAScript module syntax

---

Table of Contents

1. var name Hoisting Edge Case
2. Three Types of Node.js Modules
3. npm: The Package Manager CLI
4. package.json, node_modules, and package-lock.json
5. Local vs Global Module Installation
6. CommonJS vs ECMAScript Module Syntax
7. Combined Worked Example: ES Module HTTP Server with Express
8. Quick Revision Checklist
9. Practice Exercises
10. Practice Answers

---

var name Hoisting Edge Case

The problem this edge case creates

A student logged a variable **before** its `var` declaration and expected `undefined` (normal hoisting). Instead the console returned a **real value**. That looks like "hoisting is broken" — but the identifier was special: `name`.

Mechanism: `name` is already on `window`

In the browser, `name` **is a predefined property of the** `window` **object**. It sets or returns the name of the browser's current browsing context. Because that property already exists on `window`, treating `name` like an ordinary local `var` is misleading:

- Logging `name` before your `var name = …` line does **not** behave like a freshly hoisted binding that is still `undefined`.
- The identifier resolves against the existing `window.name` property.
- This special case is **strictly limited to the identifier** `name`. Common names like `x`, `y`, `z`, or `xyz` follow normal `var` hoisting: log-before-declaration → `undefined`.

Background: Under classic browser globals, undeclared / outer lookups can resolve on `window`. `var` at top level also attaches to `window` for many identifiers — but `name` was **already** a built-in

browsing-context property, so you are not starting from a blank slate the way you are with `xyz`.

Lecture's live console demonstration

Instructor showed this contrast in the browser console:

Code / action	Result
`var name = 'ABC'` then inspect / log `name`	Resolves to `window.name` → logs `ABC`
Log `xyz` **before** `var xyz = 'ABC'`	`undefined` (normal hoisting for a non-window special property)
Same "log before declare" pattern with other ordinary names	Always `undefined` for non-`window` specials

Memorable framing from the session: the surprising value is **not** proof that hoisting "skipped" `undefined` for every variable — it is proof that `name` **was never a normal empty binding**.

> **Exam:** `console.log(name)` before declaration can show a value because `name` is `window.name`, not because `var` hoisting invents a value for every identifier.

> **Pitfall:** Generalising "I logged before declare and got a value" to all variables. Only `name` (among the common demo names) behaves this way; `var x` / `var y` / `var z` still show `undefined` when logged early.

Exam-ready takeaway

- `window.name`: name of the current browsing context; already declared on `window`.
- `var name`: interacts with that existing property — do not use it to "prove" general hoisting rules.
- **Ordinary identifiers**: log-before-`var` → `undefined`.

Three Types of Node.js Modules

The problem "module types" answers

Node apps do not invent every helper from scratch. Some APIs ship with Node, some you write yourself, and most real features come from packages other organisations publish. Without a clear taxonomy, students cannot answer: *Do I install this? Is it already there? Who maintains it?*

Mechanism: three categories

Type	Who builds it?	How do you get it?	Examples from the lecture
Predefined (built-in)	Node.js itself	Installed automatically when Node is installed; **no separate install**	`fs`, `http`, `os`, `path`, `crypto`
User-defined	You (the project developer)	Live in your project by design	Your own `myModule.js`, helpers, utilities
Third-party	A separate external organisation	**Must be installed manually** (npm); Node cannot pre-bundle thousands of possible packages	React (Meta), Angular (Google), Express, Axios, MySQL client

Why third-party modules dominate real apps

- Predefined modules give **limited** functionality compared with what production apps need.
- Real-world applications commonly rely on **50–100+** third-party modules — far more than the built-in set.
- Most application logic therefore depends on **third-party packages**, not only on `fs` / `http` / etc.

> **Exam:** Three types = predefined (built-in with Node), user-defined (yours), third-party (external; install via npm).

> **Pitfall:** Assuming "if Node is installed, Express is already there." Express is third-party; it must be installed.

---

npm: The Package Manager CLI

The problem npm solves

Thousands of third-party modules exist. Node cannot ship all of them. You need a **standard tool** to download, install, remove, and script around packages.

Mechanism: what npm is

- **Full form:** Node Package Manager.
- **CLI-only tool** — no Play Store–style GUI in the lecture's framing.
- **Installed automatically with Node.js** — no separate install step for npm itself.

- Beyond install/uninstall, npm can also: generate application builds, run tests, execute custom scripts, run Jenkins pipelines, and run code-quality checks.

Core commands demonstrated

Goal	Command
Install a package	`npm install <package-name>` or short form `npm i <package-name>`
Remove a package	`npm uninstall <package-name>`
Live demo	`npm i express` → terminal reported **"Added 65 packages"**

Background: "65 packages" for one `express` install is normal: Express depends on other packages, and npm installs the dependency tree, not only the single top-level name you typed.

> **Exam:** npm = Node Package Manager; ships with Node; primary interface is the CLI (`npm i` , `npm uninstall` , scripts, builds, tests).

---

package.json, node_modules, and package-lock.json

The problem these three artifacts solve

After the first third-party install, every Node project in the world converges on the same shape. Without that shared structure, teammates cannot reproduce dependencies, and "it works on my machine" becomes the default.

What appears after the first `npm install` of a third-party module

Three artifacts are **always** created on the **first** third-party module installation:

1. **`node_modules/`** folder
2. **`package.json`** file
3. **`package-lock.json`** file

On **subsequent** installs, no brand-new folder/file set is invented from scratch — existing ones are **updated**.

```
project/
├── package.json          ← project config + dependency list
├── package-lock.json     ← exact resolved tree (auto-generated)
```

```
├── node_modules/          ← downloaded package source trees
└── index.js               ← your entry code (you create this)
```

package.json: purpose and contents

Mandatory in every Node.js application, even if you never install a third-party module. Skipping it means missing critical project configuration.

It is a **JSON object** (key–value pairs only; **no logic**). Fields the lecture highlighted:

Key	Role
`name`	Project name
`version`	Current version (e.g. `1.0.0`)
`description`	Short project description
`main`	Entry point file (conventionally `index.js`)
`scripts`	Commands for testing, building, deploying, quality checks
`author`	Developer / PM contact
`license`	Applicable licenses
`repository`	Git repo path so the team does not have to ask the PM
`dependencies`	Installed third-party packages and versions (**added automatically on install**)
`type`	Module system: `"commonjs"` (npm init default) or `"module"` for ES modules

Onboarding value: a new teammate reads one file and understands how the project is wired.

Creation: use `npm init`, not hand-writing from memory as the primary path. It runs an interactive CLI wizard and prompts for:

- package name (defaults to **folder name**)
- version (defaults to `1.0.0`)
- description
- entry point (defaults to `index.js`)
- test command
- Git repo
- author
- license

- module type (**CommonJS** or **ECMAScript**)

Sensible defaults are offered; you can override or skip. npm confirms the generated JSON before writing the file.

Instructor recommendation: always create `package.json` **first**, before writing code or deciding architecture.

node_modules: why it is never shared

`node_modules` stores the downloaded JS source trees for every installed third-party package.

Lecture scale numbers:

Scenario	Approximate footprint
Express + CORS alone (demo)	~**593 files**, ~**130 folders**, ~**2.17 MB**
Real app with 50–100+ packages	Often **300–700 MB**, thousands of folders, hundreds of thousands of files

Why you do **not** copy / zip / commit `node_modules` in normal practice:

- Copying or deleting a large `node_modules` can take **15–30 minutes** or more.
- Machines with limited RAM/CPU can become unresponsive during the operation.
- A single **1–2 GB** file can delete **faster** than a **500 MB** `node_modules` folder because of the **sheer number of individual files**.

Standard practice: share all source files **except** `node_modules`. Share `index.js`, `package.json`, `package-lock.json`, and application source. Exclude `node_modules` from Git for the same reason.

Recipient restore:

```
npm install
```

or `npm i` with **no** package name. npm reads the `dependencies` key in `package.json` and reinstalls everything, recreating `node_modules`.

> **Pitfall:** If a package was installed but **never recorded** under `dependencies` in `package.json`, a fresh machine's `npm install` will **not** bring it back — runtime errors follow. Always ensure every installed package has its record in `package.json`.

package-lock.json: role and rules

- **Auto-generated** — never requires manual editing.
- Maintains the **exact mapping** between each dependency listed in `package.json` and the corresponding files/folders inside `node_modules` (lecture example: maps `body-parser` to its specific folder, version, and configuration).
- Used so installs stay **consistent and reproducible** across systems and teammates.
- Not a file developers casually edit by hand day-to-day.

> **Exam:** First third-party install creates `node_modules/`, `package.json`, and `package-lock.json`. Share the JSON files + source; never share `node_modules` — recreate with `npm i`.

Local vs Global Module Installation

The problem: one machine, many projects

The same package name might be needed in project P1, P2, and P3 — or it might be a **tool** that should sit once on the machine. Installing everything the same way wastes disk and breaks resolution rules.

Local installation (default)

Aspect	Detail
Syntax	`npm i <package-name>` (no special flag)
Where it lands	Inside **that project's** folder only
Who can use it	**Only that project**; P2/P3 must install again independently
When required	Module provides a **predefined code base**: functions, classes, properties, interfaces your app **calls**
package.json	Recorded under `dependencies`
Examples	Express, CORS, Axios, bcrypt, MySQL client

These are called **dependencies** because the project's code directly calls their APIs; the project **cannot run** without them.

Instructor note: modules that provide a code base **cannot** be treated as "install once globally and require from every project" in the simple sense the lecture warns about — `require` / `import` would fail to resolve them across projects the way students often expect.

Global installation

Aspect	Detail
Syntax	`npm i -g <package-name>`
Where it lands	Node.js installation directory (e.g. `C:\Program Files\nodejs` on Windows)
Who can use it	**All projects** on the machine from that shared location
When used	Module provides a **development tool**, not an application code base
package.json	**Not** recorded as a project dependency
Examples	**nodemon**, linters, formatters, build tools, code-quality analyzers

The application can run **without** these tools in production; they support the **development workflow**.

Instructor recommendation: always install tool-type packages **globally**; installing them locally per project provides no benefit and wastes space.

Deciding factor (memorise)

Question	Install
Does the module provide functions/classes/properties **used in your code**?	**Local** → dependency
Does the module provide a **CLI / workflow tool** for development?	**Global** (`-g`)

Next-step callout from the lecture: install **nodemon** globally with `npm i -g nodemon` for automatic server restarts during development.

> **Exam:** Local = project dependency under `dependencies`. Global (`-g`) = machine-wide tool; not listed as an app dependency.

> **Pitfall:** Installing Express globally and expecting every project's `import` / `require` to "just find it." Code-base libraries belong **local**.

CommonJS vs ECMAScript Module Syntax

The problem two syntaxes create

Node historically defaulted to **CommonJS** (`require` / `exports`). Modern apps prefer **ECMAScript** modules (`import` / `export`). Mixing them without setting `package.json` `"type"` produces a hard runtime error.

Side-by-side syntax

Concern	CommonJS (legacy, still widely used)	ECMAScript (modern, recommended for new apps)
Import built-in	`const http = require('http')`	`import { createServer } from 'http'`
Import custom module	`const m = require('./myModule')`	`import { sum, product } from './myModule.js'`
Export	`exports.sum = function (a, b) { return a + b }`	`export function sum(a, b) { return a + b }`
Calling style	Often via module object (`http.createServer`)	Call imported bindings **by name** (`createServer()` , not `http.createServer()`)
`package.json` `"type"`	`"commonjs"` (**default** from `npm init`)	Must be `"module"`

With ES modules you **do not** need the `exports.propertyName = …` pattern — put `export` directly on the declaration.

Live error demo from the lecture

1. `node index.js` used `import` syntax while `package.json` still had `type: "commonjs"` (or default CommonJS behaviour) → error: `Cannot use import statement outside a module` .
2. Changed `"type"` to `"module"` → server started successfully and responded with `hello world` on **port 5000**.

Entry-point naming convention

Name	Role
`index.js` (recommended)	Treated as the welcome / starting file by convention

Name	Role
Also accepted	`app.js`, `server.js`, `main.js`

Recommendation only, not a hard language rule — `index.js` keeps the team aligned.

Migration guidance from the instructor

- **New applications:** use ECMAScript `import` / `export` exclusively; set `"type": "module"`.
- **Huge legacy codebases:** may be too large to migrate off `require`; keep CommonJS there.
- Most modern applications use **`import` / `export` only**.

> **Exam:** `"type": "module"` is required for `import` / `export` in Node; default CommonJS + `import` → *Cannot use import statement outside a module.*

> **Pitfall:** Writing ES `import` in `index.js` after a stock `npm init` without changing `"type"` from `"commonjs"`.

Combined Worked Example: ES Module HTTP Server with Express

This end-to-end mini-project ties together **`npm init`**, **`type: "module"`**, a **local** Express dependency, **`package.json` / lockfile / `node_modules`**, and ES **`import`** — the practical arc of the morning.

Step trace

1. Create a project folder and run `npm init` (accept defaults, but set module type to **ECMAScript** / ensure `"type": "module"`).
2. Install Express **locally**: `npm i express` → creates/updates `node_modules`, `package.json` `dependencies`, and `package-lock.json`.
3. Write `index.js` with ES imports (no `require`).
4. Run `node index.js` and open `http://localhost:5000`.

`package.json` (illustrative after init + Express install)

```
{
  "name": "day7-morning-demo",
  "version": "1.0.0",
  "description": "WPT Day 7 Morning combined example",
```

```
  "main": "index.js",
  "type": "module",
  "scripts": {
    "start": "node index.js"
  },
  "author": "",
  "license": "ISC",
  "dependencies": {
    "express": "^4.21.0"
  }
}
```

`index.js` (copy-paste runnable once Express is installed)

```
import express from 'express';

const app = express();
const PORT = 5000;

app.get('/', (req, res) => {
  res.send('hello world');
});

app.listen(PORT, () => {
  console.log(`Server listening on http://localhost:${PORT}`);
});
```

End-to-end behaviour

Step	What happens
`npm i express`	Express + its dependency tree land in `node_modules`; versions recorded in `package.json` / lockfile
`"type": "module"`	Node treats `.js` files as ES modules → `import` is legal
`import express from 'express'`	Resolves the **local** dependency (not a global tool)
Browser → `http://localhost:5000/`	Response body `hello world` (same idea as the lecture's port-5000 demo)
Teammate clones without `node_modules`	Runs `npm i` → tree restored from `package.json` / lockfile

If `"type"` were left as `"commonjs"`, the same `import` line would throw **Cannot use import statement outside a module** — exactly as demonstrated live.

Quick Revision Checklist

Topic	One-line takeaway
`var name` edge case	Early `name` can show a value because of `window.name`, not because every `var` skips `undefined`.
Module types	**Predefined** (with Node), **user-defined** (yours), **third-party** (install via npm).
npm	**Node Package Manager** CLI; ships with Node; `npm i` / `npm uninstall` (+ scripts, builds, tests).
First install artifacts	Creates `node_modules/`, `package.json`, `package-lock.json`.
`package.json`	Mandatory project config JSON; create with `npm init` before coding.
`node_modules`	Huge file/folder tree — **never share/commit**; recreate with `npm i`.
`package-lock.json`	Auto-generated exact dependency mapping; do not hand-edit.
Local vs global	Code libraries → **local** + `dependencies`; CLI tools → `npm i -g`.
CommonJS vs ES	`require` / `exports` vs `import` / `export`; ES needs `"type": "module"`.
Entry file	Prefer `index.js` (also `app.js` / `server.js` / `main.js`).

Practice Exercises

Concept

1. Why can `console.log(name)` before a `var name` declaration show a real string in the browser, while `console.log(xyz)` before `var xyz` shows `undefined`?
2. Name the three types of Node.js modules and give one example of each from the lecture.
3. What three artifacts appear after the **first** third-party `npm install` in a project?
4. Why is `node_modules` excluded from Git even though the app cannot run without those packages present locally?
5. How do you decide whether to install a package locally or globally?

Application

6. Write the exact npm commands to (a) create `package.json` interactively, (b) install Express locally, (c) install nodemon globally.
7. A teammate sends you `index.js`, `package.json`, and `package-lock.json` but not `node_modules`. What single command restores dependencies, and what does npm read to know what to install?
8. Set up the minimum `package.json` field change needed so this file runs under Node:

```
import { createServer } from 'http';
createServer((req, res) => {
  res.end('hello world');
}).listen(5000);
```

Predict output / behaviour

9. `package.json` has `"type": "commonjs"`. File `index.js` contains `import express from 'express';`. You run `node index.js`. What happens?
10. You ran `npm i cors` on your laptop but somehow `cors` never appeared under `dependencies` in `package.json`. A colleague clones the repo and runs `npm i`. Will `cors` be installed? What goes wrong later?

Spot the bug

11. A student installs Express with `npm i -g express` in order to use `import express from 'express'` inside Project A. What misconception is this, and what should they do instead?
12. Find two problems in this beginner workflow description: "I skipped `npm init`, wrote `index.js` with `require('http')`, then later switched the file to `import` syntax without touching any JSON file, and I committed `node_modules` so my friend would not need npm."

Practice Answers

1.

`name` is a predefined property of the browser `window` object (`window.name` = browsing-context name). Logging it early can resolve to that existing property / assignment path. `xyz` is an ordinary identifier, so classic `var` hoisting yields `undefined` before the declaration runs.

2.

Predefined: e.g. `fs`, `http`, `os`, `path`, `crypto`. User-defined: modules you write in the project. Third-party: e.g. React, Angular, Express, Axios, MySQL client (installed via npm).

3.

`node_modules/` folder, `package.json`, and `package-lock.json`.

4.

It is enormous (many tiny files/folders; hundreds of MB in real apps). Copy/delete/commit is slow and painful; standard practice is to share `package.json` (+ lockfile) and let recipients run `npm i` to recreate it.

5.

If the package provides functions/classes/APIs your application code calls → **local** (dependency). If it is a development CLI/tool (nodemon, linters, formatters, build tools) → **global** with `-g`.

6.

```
npm init
npm i express
npm i -g nodemon
```

7.

Command: `npm install` or `npm i` (no package name). npm reads the `dependencies` (and the lockfile for exact versions) from `package.json` / `package-lock.json` and recreates `node_modules`.

8.

In `package.json`, set:

```
"type": "module"
```

(and keep using ES `import` syntax). Otherwise Node treats the file as CommonJS and rejects `import`.

9.

Node throws an error along the lines of **Cannot use import statement outside a module** (as in the live demo). Fix: set `"type": "module"` (or rename to `.mjs` in ecosystems that use that convention — the lecture's fix was changing `type` to `"module"`).

10.

No — a bare `npm i` only installs what is listed. Missing `cors` in `dependencies` means it will not be restored; the colleague's app will fail when code tries to load `cors`. Always keep `package.json` in sync with installs.

11.

Express is a **code-base library** (dependency), not a global CLI tool. Global install will not correctly

serve as “the project dependency” the way the lecture describes for `import` / `require` across projects. Install **locally**: `npm i express` so it appears under `dependencies` and in that project’s `node_modules`.

12.

(1) Switching to `import` without setting `"type": "module"` in `package.json` causes the CommonJS/ES module error. (2) Committing / sharing `node_modules` is the wrong distribution strategy — share manifests and run `npm i`. Skipping `npm init` also fights the instructor’s “create `package.json` first” rule and leaves you without a proper dependency record.